

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TUẦN 14
MÔN THỰC TẬP CƠ SỞ

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

MỤC LỤC

1. Mục tiêu công việc trong tuần	3
2. Nội dung công việc đã thực hiện	3
2.1 Tối ưu hóa tốc độ với ONNX (export_onnx.py)	3
2.2 Xây dựng máy chủ xử lý độc lập (api.py)	3
2.3 Tích hợp AI giải thích - Grad-CAM (gradcam_helper.py)	3
2.4 Xây dựng trợ lý chuyên gia LLM (gemini_helper.py)	3
2.5 Tái cấu trúc và tối ưu hóa mã nguồn giao diện (Refactoring).....	3
3. Kết quả đạt được	4
4. Khó khăn gặp phải	4
5. Kế hoạch tuần tiếp theo	4

1. Mục tiêu công việc trong tuần

Trong tuần trước, hệ thống đã được gia cố tính toàn vẹn dữ liệu và ngăn chặn rủi ro tràn bộ nhớ. Bước sang tuần 14, mục tiêu của em chuyển hướng sang việc **tối ưu hóa hiệu năng hệ thống (System Optimization)** và **tăng tính minh bạch của AI**. Nhiệm vụ trọng tâm bao gồm nén mô hình, xây dựng kiến trúc máy chủ độc lập (Backend), tích hợp hệ thống chuyên gia tư vấn (LLM) và tái cấu trúc (Refactoring) mã nguồn giao diện.

2. Nội dung công việc đã thực hiện

2.1 Tối ưu hóa tốc độ với ONNX (export_onnx.py)

Để giải quyết tình trạng mô hình PyTorch (.pth) gốc khá nặng và chạy chậm trên CPU, em đã viết một đoạn mã (script) chuyển đổi toàn bộ kiến trúc mạng sang định dạng chuẩn ONNX. Việc ép nén này giúp hệ thống phản hồi kết quả gần như ngay lập tức mà không cần phụ thuộc vào Card đồ họa (GPU) cấu hình cao.

2.2 Xây dựng máy chủ xử lý độc lập (api.py)

Em đã tiến hành tách lõi nhận diện AI ra khỏi giao diện Streamlit bằng cách xây dựng một máy chủ Backend sử dụng thư viện **FastAPI**. Giao diện Web giờ đây chỉ làm nhiệm vụ hiển thị, còn toàn bộ các tính toán nặng nề được đẩy sang API xử lý ngầm, giúp Web không bao giờ bị đơ (freeze) trong quá trình người dùng tải ảnh.

2.3 Tích hợp AI giải thích - Grad-CAM (gradcam_helper.py)

Để giúp người dùng hiểu tại sao AI lại đưa ra quyết định phân loại, em đã tích hợp thuật toán Grad-CAM. Thuật toán này tự động trích xuất các đặc trưng và vẽ một "Bản đồ nhiệt" (Heatmap) đè lên bức ảnh gốc, làm nổi bật vùng pixel (màu đỏ) mà mô hình đang tập trung nhìn vào.

2.4 Xây dựng trợ lý chuyên gia LLM (gemini_helper.py)

Em tích hợp thêm API của Google Gemini để đóng vai trò chuyên gia tư vấn. Hệ thống sẽ cung cấp cho Gemini thông tin về nhãn rác và bản đồ nhiệt. Từ đó, Gemini tự động quan sát tình trạng rác (ví dụ: chai nhựa này sạch hay bẩn, hộp giấy có dính mỡ không) để đưa ra lời khuyên vứt rác/tái chế cụ thể bằng văn bản.

2.5 Tái cấu trúc và tối ưu hóa mã nguồn giao diện (Refactoring)

Nhận thấy tính năng hiển thị đa ngôn ngữ (I18N) làm cấu trúc mã nguồn rườm rà và không thực sự cần thiết cho nghiệp vụ cốt lõi, em đã tiến hành gỡ bỏ hoàn toàn hệ

thông này. Toàn bộ giao diện (app.py, page_scanner.py, page_dashboard.py) và các kịch bản tư vấn (Prompt) được chuẩn hóa về dạng thuần Tiếng Việt. Điều này giúp mã nguồn giảm phân nửa dung lượng, cực kỳ tinh gọn, dễ bảo trì và loại bỏ được các lỗi hiển thị (UI bugs) không đáng có.

3. Kết quả đạt được

Hệ thống hiện tại đã hoạt động theo kiến trúc phân tán (Client-Server) rất chuyên nghiệp. Mô hình ONNX kết hợp FastAPI mang lại tốc độ nhận diện siêu tốc. Việc bổ sung Grad-CAM và Gemini giúp ứng dụng không chỉ phân loại "máy móc", mà còn biết giải thích trực quan và tư vấn chi tiết như một con người thực thụ. Mã nguồn giao diện đạt trạng thái sạch sẽ (Clean Code) 100%.

4. Khó khăn gặp phải

Khó khăn: Nghẽn mạng khi gọi API bên thứ ba làm treo hệ thống

- **Mô tả vấn đề:** Khi đưa trợ lý Gemini vào hoạt động, em nhận thấy đôi lúc máy chủ Cloud của Google phản hồi rất chậm. Giao diện Web phải chờ đợi vô thời hạn (loading liên tục) khiến trải nghiệm người dùng bị gián đoạn.
- **Quá trình phân tích và Khắc phục:** Em nhận ra rằng không thể tin tưởng tuyệt đối vào kết nối mạng bên ngoài. Qua tìm hiểu, em đã áp dụng phương pháp lập trình phòng thủ: Cài đặt biến timeout giới hạn thời gian chờ của API. Nếu quá 15 giây mà Gemini không trả lời, hệ thống sẽ bắt (catch) lỗi này và tự động chuyển sang sử dụng "Từ điển ngoại tuyến (Offline Dictionary)" nội bộ do em tự định nghĩa trước đó để tư vấn cho người dùng.
- **Bài học rút ra:** Mọi tính năng phụ thuộc vào Internet đều cần có cơ chế ngắt tự động (timeout) và phương án dự phòng (fallback) để bảo vệ luồng hoạt động cốt lõi của phần mềm.

5. Kế hoạch tuần tiếp theo

Tuần 15 là tuần cuối cùng của kỳ thực tập cơ sở. Toàn bộ thời gian sẽ được dành cho việc tối ưu lại độ chính xác cốt lõi của mô hình AI và đóng gói dự án:

- **Làm sạch dữ liệu và Huấn luyện lại (Retrain):** Phân tích và quyết định loại bỏ lớp rác trash (rác hỗn hợp) ra khỏi tập dữ liệu vì đặc trưng hình dáng không rõ ràng gây nhiễu cho mô hình. Tiến hành huấn luyện lại mạng EfficientNet với 9 lớp chuẩn để nâng độ chính xác thực tế lên mức tối đa.

- **Tổng duyệt hệ thống (End-to-End Testing):** Chạy thử nghiệm toàn bộ luồng chức năng với mô hình AI 9 lớp mới để đảm bảo tính đồng bộ giữa Frontend và Backend.
- **Hoàn thiện Báo cáo:** Bắt tay vào viết quyền báo cáo tổng kết môn học (Word/PDF) và thiết kế slide thuyết trình (PowerPoint).